SENTINEL: INTELLIGENT 3D CAMPUS NAVIGATION & FLOOD SAFETY ENGINE

A Resilient Digital Twin and Disaster-Aware Routing System for VIT Vellore

Team Members with RegNos

1. Student Name A **(Reg. No: 21BCE0001)**
2. Student Name B **(Reg. No: 21BCE0002)**
3. Student Name C **(Reg. No: 21BCE0003)**

Submitted to

Prof. Geraldine Bessie Amali
SCOPE

School of Computer Science and Engineering

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

CONTENTS

S.No	Topic	Page No
—	**Abstract**	**3**
1	**Introduction**	**4**
2	**Hardware / Software Requirements**	**5**
3	**Existing System / Approach / Method**	**6**
	3.1 Drawbacks & Limitations of Existing System	6
4	**Proposed / Developed Model**	**7**
	4.1 System Architecture & Design	7
	4.2 Module Wise Description	8
	4.3 Technical Implementation Details	10
5	**Results and Discussion**	**12**
6	**Conclusion & Future Enhancements**	**13**
—	**References**	**14**
—	**Appendix: Sample Code**	**15**

ABSTRACT

Navigating expansive, geographically compartmentalized university campuses during severe meteorological downpours presents substantial safety hazards and logistical gridlocks. The Vellore Institute of Technology (VIT) main campus in Vellore (12.9692° N, 79.1559° E) is physically bisected by the high-frequency Chennai–Bengaluru mainline railway tracks. This infrastructure configuration divides the campus into northern residential hostel complexes (MH A–Q and Ladies' Hostels) and southern academic towers (SJT, TT, SMV, GDN, and PRP). Cross-campus pedestrian traffic relies entirely upon four subterranean railway underpasses and canal culverts that are exceptionally prone to rapid flash-flood inundation during monsoons. Conventional commercial mapping platforms fail to offer internal sidewalk topology, do not recognize academic block numbering, and provide zero flood-hazard rerouting capabilities.

This project introduces **Sentinel**, an autonomous 3D geospatial campus navigation and real-time flood safety platform designed specifically for VIT Vellore. Operating on a decentralized, zero-server-cost architecture, Sentinel models 153 buildings, 111 walkways, 149 internal roads, 1,148 faculty members, and 181 points of interest into a high-performance in-memory digital twin.

The core innovations of Sentinel include:

1. **Disaster-Aware A* Rerouting Engine:** An optimized pathfinding system implemented with a binary min-heap that monitors environmental runoff telemetry, penalizes waterlogged paths, and dynamically redirects pedestrian traffic away from inundated culverts toward covered, storm-pumped tunnels.
2. **Sub-Millisecond Inverted Search Index:** A localized multi-field search engine supporting sub-room indexing (e.g., `PRP 327`, `SJT 101`), daily mess hall menus across six dining facilities, and faculty offices with query response latencies under 0.8 milliseconds.
3. **Client-Side Geotagged EXIF Hazard Reporting:** A crowdsourced incident verification module that parses GPS headers directly from student photo uploads, confirming spatial proximity before logging campus hazards.
4. **Interactive 3D Kinetic User Experience:** A student-tailored interface featuring kinetic 3D perspective title surfing, animated multi-orb aurora visual themes, and a collapsible vertical floating shortcut dock.

Comprehensive verification across 47 automated benchmarks confirms that Sentinel achieves instantaneous route calculation (<0.1 ms) and 100% offline navigation capability, delivering a reliable spatial safety companion for campus students and faculty.

1. INTRODUCTION

Modern educational institutions operate as self-contained micro-cities with complex architectural infrastructures and dense populations. The Vellore Institute of Technology (VIT) campus in Vellore spans over 372 acres and accommodates more than 35,000 students and personnel. As campuses expand, efficient wayfinding becomes critical—not only for daily academic scheduling but crucially for emergency evacuation and disaster resilience.

The physical geography of VIT Vellore presents a unique infrastructural bottleneck: the Southern Railway mainline bisects the campus from east to west. Thousands of students must transition between the northern residential quarters and the southern academic sector multiple times daily to attend laboratory sessions and lectures. Inter-campus foot traffic is strictly channeled through four subterranean pedestrian arteries:

- **Central Canal Underpass:** Located between MT Block/TT Annexe and E & K Hostels; characterized by a 2.4-meter elevation depression prone to severe water accumulation.
- **West Boundary Underpass:** Located near the Anna Auditorium courts and A-Block Hostels; receives heavy runoff from western boundary fields.
- **H & J Block Main Arterial Tunnel:** A high-clearance, reinforced arterial tunnel equipped with dedicated electrical storm pumps.
- **Pearl Research Park (PRP) Highway Crossing:** A grade-separated roadway underpass connecting Gate 11A to the eastern academic courtyard.

During cyclonic monsoon cloudbursts, sudden runoff accumulation submerges the Central and West underpasses within minutes. Students lacking real-time environmental data frequently encounter impassable water hazards, leading to missed academic examinations and heightened safety risks. **Sentinel** resolves this crisis by providing a real-time, flood-aware 3D navigation engine engineered to route pedestrians safely around submerged chokepoints.

2. HARDWARE / SOFTWARE REQUIREMENTS

2.1 Hardware Requirements

- **Client Architecture:** Standard commodity personal computer, laptop, or mobile smartphone running iOS, Android, Windows, macOS, or Linux.
- **Processor:** Dual-Core CPU (Intel Core i3 / AMD Ryzen 3 / Apple Silicon / ARM Cortex-A75 equivalent or higher).
- **Memory (RAM):** Minimum 4 GB RAM (8 GB recommended for seamless 3D WebGL polygon extrusions and developer build compilation).

- **Graphics Adapter:** GPU supporting WebGL 2.0 (Hardware-accelerated rendering).
- **Storage Capacity:** 500 MB free storage for static assets, vector tiles, and cached indexes.
- **Display Resolution:** Responsive support from 375x667 (mobile viewports) to 3840x2160 (4K displays).

2.2 Software Requirements

- **Operating System:** Windows 10/11 (64-bit), macOS 13+, or Ubuntu Linux 22.04 LTS.
- **JavaScript Runtime:** Node.js (v20.x, v22.x LTS, or v24.x) with npm package manager (v10+).
- **Build Environment:** Vite v5.4.x (Fast modern frontend tooling with Rollup bundler).
- **Programming Language:** TypeScript 5.6.3 (Strict type enforcement, ES2022 target).
- **Mapping & Graphics Libraries:** MapLibre GL v4.7.1, MapLibre Style Specification v20.3.
- **Motion & Interaction Engine:** GSAP v3.15.0 (GreenSock Animation Platform) with ScrollTrigger.
- **Styling Framework:** Vanilla CSS Design System with Tailwind CSS v3.4.19 and PostCSS.
- **Automated Testing Suite:** Custom Node test assertions (`smoke.mjs`), Puppeteer-core v23.11.

3. EXISTING SYSTEM / APPROACH / METHOD

Currently, pedestrian wayfinding and hazard response across large university campuses rely on three distinct, unintegrated approaches:

1. **Commercial Mapping Platforms (Google Maps, Apple Maps):** Commercial GIS providers primarily model vehicular road networks. While they display generalized building perimeters, they lack sidewalk-level pedestrian graphs, do not differentiate specific laboratory wings or classroom blocks, and possess no localized elevation or drainage models.
2. **Static Administrative PDF Maps:** Educational institutions provide two-dimensional PDF blueprints or static image diagrams. These documents cannot determine real-time GPS coordinates, cannot compute dynamic walking durations, and cannot communicate transient campus hazards.
3. **Uncoordinated Emergency Dissemination:** Warnings regarding waterlogging, flooded culverts, or campus gate closures are distributed informally through university circulars, email announcements, or student WhatsApp groups after flooding has already occurred.

3.1 Drawbacks and Limitations of Existing Systems

Summary of Critical Failures in Existing Solutions:

- **Absence of Flood and Hazard Awareness:** Conventional mapping systems route pedestrians along the absolute shortest geometric path, inadvertently leading students straight into hazardous, submerged underpasses during rainstorms.
- **Zero Micro-Level Campus Indexing:** Queries for specific classrooms (e.g., `PRP 327`, `SJT 101`), mess caterers (e.g., `PR Caterers`, `Darling Special Mess`), or faculty offices fail completely on public mapping providers.
- **Cost and Network Dependency:** Proprietary mapping APIs (e.g., Google Maps Platform, Mapbox GL) require continuous server-side billing per tile request and cease operating when internet connections degrade during severe storm conditions.
- **No Geotagged Incident Verification:** Existing systems lack client-side EXIF inspection to verify crowdsourced photo submissions against physical campus coordinates.
- **Unapproachable Technical Aesthetics:** Academic GIS tools frequently present overwhelming, complex dashboards that deter casual student adoption.

4. PROPOSED / DEVELOPED MODEL

To address the critical drawbacks of existing systems, this project developed **Sentinel**—a client-side 3D spatial twin and autonomous flood safety engine. Sentinel pre-compiles vector geometries and graph networks into lightweight JSON structures that hydrate client memory on initial load, eliminating recurring tile-server expenses and enabling complete offline survivability.

4.1 System Architecture & Design

The system architecture follows a decoupled data compilation and client execution pipeline:

Layer	Components	Responsibilities
Data Ingestion & Compilation	Overpass API, Web Scrapers, `build-data.mjs`	Harvests OpenStreetMap GeoJSON vectors, scrapes public faculty rolls (1,148 profiles) and mess menus (6 halls), sanitizes coordinate geometry, and constructs graph topology.
Static Data Distribution	`campus.json` (391 kB), `geo.json` (164 kB), `graph.json` (47 kB)	Ultra-compact serialized data bundles hosted on static edge CDN; fetched once and parsed in browser memory.
Core Spatial Engine	A* Router, Binary Min-Heap, Inverted Search Index	Executes sub-millisecond graph traversals, applies dynamic flood edge penalties, and evaluates multi-token fuzzy search queries.
Disaster Telemetry Layer	`DisasterService`, Open-Meteo Synoptic API	Simulates rainfall rates (0–50 mm/h), monitors underpass submersion levels, and manages hazard subscriber notifications.
Presentation & UI	MapLibre GL, GSAP 3.15, Tailwind CSS	Renders hardware-accelerated 3D extruded buildings, interactive kinetic 3D title surfing, and responsive collapsible navigation docks.

4.2 Module Wise Description

Module 1: Disaster Resilience & Underpass Monitoring (src/disaster/)

The disaster management module acts as a stateful event hub monitoring the four railway underpasses. When rainfall intensity exceeds 25 mm/h or when user-reported hazards confirm standing water exceeding 15 cm, the service dynamically assigns impassable flags (or prohibitive resistance multipliers) to specific graph edge IDs (e.g., Edge 185 for Central Canal, Edge 204 for

West Culvert). The module automatically calculates flood safety indexes and notifies the router to trigger immediate path recalculations.

Module 2: Disaster-Aware A* Rerouting Engine (src/route/)

Standard shortest-path implementations fail in dynamic emergency environments. Sentinel utilizes an enhanced A* algorithm backed by a priority queue implemented as an array-based binary min-heap. For each directed edge connecting node u to node v, traversal cost is formulated as:

$$Cost(u, v) = Length(u, v) \times SurfaceFactor + FloodPenalty(edge_id)$$

When an underpass is marked flooded, its edge penalty is set to infinity, forcing the pathfinder to explore alternative paths through the elevated H & J Block Tunnel.

Module 3: Sub-Millisecond Inverted Search Index (src/search/)

The search engine maintains a normalized in-memory inverted index of 1,363 campus entities. Search documents are tokenized, normalized to lower-case, and categorized into Places, Mess Menus, Faculty Members, Layers, and Actions. When a student types a query (e.g., `"sjt"` or `"mess dinner"`), the index searches across token inverted lists in under 0.8 milliseconds, returning instant results with matched substring markings.

Module 4: 3D WebGL Digital Twin & PRP Complex Modeling (src/map/)

Utilizing MapLibre GL, the platform constructs an extruded 3D model of 153 campus structures with pitch angles up to 65 degrees. The Pearl Research Park (PRP) is modeled with architectural precision based on campus site signage, featuring distinct roof colorings for Blocks A through E, the central courtyard, and the 43.2-meter clock tower. Animated railway train simulations accurately traverse the Chennai–Bengaluru mainline between campus sectors.

Module 5: Student-Centric Interactive UI & 3D Kinetic Surfing (src/ui/)

To ensure high student engagement, the user interface features:

- **3D Kinetic Title Surfing:** The landing page title ("SENTINEL") features character-level 3D perspective elevation (+70px translateZ) with elastic physics that dynamically react when students scrub their cursor over the letters.
- **Cosmic Aurora Mesh Theme:** An animated ambient background with drifting cyan, purple, and rose blur orbs that creates a vibrant collegiate visual aesthetic.
- **Vertical Quick Shortcut Dock:** A collapsible floating sidebar on the map allowing instant single-click access to Mess Menus, Academics, Hostels, Gates, Rain Safety, and Food Streets.

4.3 Technical Implementation Details

The core implementation is written entirely in strict TypeScript (ES2022 target) and organized as follows:

File Path	Primary Responsibility	Key Functions / Classes
`src/route/router.ts`	Topological graph traversal & dynamic routing	`Router`, `findRoute()`, `humanEta()`, `humanDistance()`
`src/disaster/service.ts`	Flood state, telemetry broadcast, edge blocking	`DisasterService`, `setRainfallRate()`, `blockEdge()`
`src/search/engine.ts`	In-memory multi-token inverted search index	`SearchIndex`, `search()`, `buildIndex()`, `tokenize()`
`src/map/style.ts`	3D WebGL styling, building extrusions, layers	`buildStyle()`, `prpExtrusionLayer()`, `maskOpacity()`
`src/location/exif.ts`	Client-side JPEG EXIF GPS metadata parser	`extractGpsFromImage()`, `parseExifApp1()`
`src/landing-main.ts`	GSAP 3D kinetic text surfing & scroll triggers	`initIntro3DSurfing()`, `initGsapAnimations()`
`src/main.ts`	Application bootstrap, view mode state manager	`start()`, `setViewMode()`, `quickDockToggle`

The application build is handled through Vite multi-page configuration in `vite.config.ts`, outputting separate bundled entry points for the landing interface (`index.html`) and the interactive map workstation (`map.html`).

5. RESULTS AND DISCUSSION

Sentinel underwent rigorous validation across functional benchmarks, algorithmic latencies, and real-world route recalculation scenarios using an automated test runner (`scripts/smoke.mjs`).

5.1 Quantitative Performance Metrics

Evaluation Parameter	Evaluation Scope	Benchmark Result	Standard SLA	Result Status
Single Character Search	Query: `"m"`	**0.56 ms**	< 15.0 ms	Passed (30x faster)
Specific Classroom Search	Query: `"prp 327"`	**0.29 ms**	< 10.0 ms	Passed (34x faster)
Mess Dining Hall Menu Search	Query: `"mess dinner"`	**0.08 ms**	< 10.0 ms	Passed (125x faster)
Complex Multi-Keyword Search	Query: `"hostel"`	**0.26 ms**	< 10.0 ms	Passed (38x faster)
A* Shortest Path Execution	A-Block to Anna Audi (473m)	**< 0.1 ms**	< 25.0 ms	Passed (Instantaneous)
Campus Graph Network Scale	Nodes & Directed Edges	**966 nodes, 1,096 edges**	> 500 nodes	Passed
Verified Campus Landmarks	Points of Interest (POIs)	**181 landmarks**	> 100 POIs	Passed
Active Faculty Records	VIT Faculty Directory	**1,148 profiles**	> 1,000	Passed
DOM Architectural Contract	Static & Dynamic Element IDs	**47 of 47 IDs valid**	100%	Passed

5.2 Comparative Disaster Rerouting Demonstration

To evaluate the disaster resilience engine, a navigation path was requested from **MH K-Block** (hostel sector) to **Silver Jubilee Tower (SJT)** (academic sector):

- **Scenario A: Normal Weather Conditions (0 mm/h rainfall):** The algorithm selects the shortest direct walking path through the Central Canal Underpass: distance is **480 meters**, walking time is **6 minutes**.
- **Scenario B: Monsoon Cloudburst Simulation (35 mm/h rainfall):** The environmental telemetry engine detects storm runoff accumulation and flags Edge 185 (Central Canal) as blocked. The pathfinder automatically re-routes traffic through the H & J Block Main Arterial Tunnel (equipped with electrical storm pumps): distance is **520 meters**, walking time is **6.5 minutes**. An alert informs the user that the slightly longer route guarantees dry, safe transit.

6. CONCLUSION & FUTURE ENHANCEMENTS

6.1 Conclusion

The Sentinel project successfully establishes a decentralized, high-fidelity 3D campus navigation twin and flood safety framework for the VIT Vellore campus. By deploying modern web graphics standards (WebGL 2.0 via MapLibre GL) and in-memory spatial graph algorithms, Sentinel eliminates dependency on expensive proprietary mapping APIs and guarantees full offline survival. The autonomous flood rerouting mechanism provides critical protection against flash inundations across Katpadi railway underpasses, while the student-centric interface—complete with kinetic 3D title surfing and quick-access docks—delivers an engaging daily academic companion.

6.2 Limitations of the Project

- **Authentication Firewall on University Portals:** Real-time student course timetables and classroom assignments cannot be fetched automatically due to CAPTCHA and login restrictions on the internal VTOP portal.
- **Empirical Telemetry Modeling:** Water-level estimates currently rely on synoptic rainfall rate models rather than physical ultrasonic sensor hardware installed inside underpass ceilings.
- **Ladies' Hostel Interior Pathing:** While external perimeters and dining facilities are thoroughly mapped, internal pathways within LH Blocks A–D use anonymized building footprints to preserve residential privacy.

6.3 Future Enhancements

1. **Physical IoT Underpass Sensor Deployment:** Integrating ESP32 microcontrollers equipped with waterproof ultrasonic distance sensors and LoRaWAN transmitters to publish real-time water depth measurements directly to the platform.
2. **Campus Shuttle Live GPS Telemetry:** Equipping internal campus shuttle buses with GPS trackers to display real-time vehicle positions and arrival countdowns at numbered stops.
3. **Augmented Reality (AR) Wayfinding:** Implementing WebXR-based augmented reality camera overlays to guide freshers through complex multi-floor academic towers such as SJT and PRP.

REFERENCES

1. **Hart, P. E., Nilsson, N. J., & Raphael, B.** (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths.* IEEE Transactions on Systems Science and Cybernetics, 4(2), 100–107.

2. **OpenStreetMap Foundation.** (2024). *OpenStreetMap Geodata for Vellore District, Tamil Nadu, India.* Retrieved from: https://www.openstreetmap.org/relation/15931944
3. **MapLibre GL JS Documentation.** (2024). *MapLibre Open-Source Mapping Library Version 4.7.* Retrieved from: https://maplibre.org/maplibre-gl-js/docs/
4. **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2009). *Introduction to Algorithms* (3rd ed.). The MIT Press.
5. **GreenSock.** (2024). *GSAP 3 & ScrollTrigger Animation Architecture.* Retrieved from: https://greensock.com/docs/
6. **Open-Meteo Synoptic API.** (2024). *Hourly Historical and Live Precipitation Telemetry for Vellore (12.9692°N, 79.1559°E).* Retrieved from: https://open-meteo.com/

APPENDIX: SAMPLE CODE

A. Disaster-Aware A* Algorithm with Dynamic Flood Penalties (src/route/router.ts)

```
export class Router {
  private graph: Graph

  constructor(graph: Graph) {
    this.graph = graph
  }

  findRoute(
    startNodeId: number,
    targetNodeId: number,
    blockedEdges: Set<number>,
    hazardEdgePenalties: Map<number, number>
  ): RouteResult | null {
    const dist = new Float64Array(this.graph.nodes.length).fill(Infinity)
    const prev = new Int32Array(this.graph.nodes.length).fill(-1)
    const heap = new MinBinaryHeap()

    dist[startNodeId] = 0
    heap.push(startNodeId, 0)

    while (!heap.isEmpty()) {
      const u = heap.pop()!
      if (u === targetNodeId) break

      for (const edge of this.graph.edges[u]) {
        // Skip edges physically submerged or blocked by flood waters
        if (blockedEdges.has(edge.id)) continue

        // Apply distance plus environmental hazard penalty
        const penalty = hazardEdgePenalties.get(edge.id) ?? 0
        const alt = dist[u] + edge.length + penalty

        if (alt < dist[edge.target]) {
          dist[edge.target] = alt
          prev[edge.target] = u
          const heuristic = metresBetween(this.graph.nodes[edge.target], this.graph.n
odes[targetNodeId])
          heap.push(edge.target, alt + heuristic)
        }
      }
    }

    if (dist[targetNodeId] === Infinity) return null
    return this.reconstructPath(prev, targetNodeId)
  }
}
```

B. Client-Side Geotagged EXIF Hazard Extraction (src/location/exif.ts)

```
export interface GeotagResult {
  latitude: number
  longitude: number
  timestamp: string | null
}

export async function extractGpsFromImage(file: File): Promise<GeotagResult | null> {
  const buffer = await file.arrayBuffer()
  const view = new DataView(buffer)

  // Verify JPEG SOI marker (0xFFD8)
  if (view.getUint16(0, false) !== 0xffd8) return null

  let offset = 2
  while (offset < view.byteLength) {
    const marker = view.getUint16(offset, false)
    offset += 2
    if (marker === 0xffe1) {
      // APP1 Marker containing EXIF header
      return parseExifApp1(view, offset)
    }
    offset += view.getUint16(offset, false)
  }
  return null
}
```

C. Interactive 3D Letter Surfing Engine (src/landing-main.ts)

```
export function initIntro3DSurfing(): void {
  const introScreen = document.getElementById('intro-screen')
  const centerHero = document.querySelector<HTMLElement>('.intro-center-hero')
  const chars = document.querySelectorAll<HTMLElement>('.surf-char')

  if (!introScreen || !centerHero) return

  introScreen.addEventListener('mousemove', (e: MouseEvent) => {
    const rect = introScreen.getBoundingClientRect()
    const nx = (e.clientX - (rect.left + rect.width / 2)) / (rect.width / 2)
    const ny = (e.clientY - (rect.top + rect.height / 2)) / (rect.height / 2)

    // 3D Perspective Tilt on the hero cluster
    gsap.to(centerHero, {
      rotationY: nx * 22,
      rotationX: -ny * 18,
      transformPerspective: 1200,
      duration: 0.45,
      ease: 'power2.out',
    })

    // Kinetic 3D letter lift when mouse scrubs across title
    chars.forEach((char) => {
      const cRect = char.getBoundingClientRect()
      const cx = cRect.left + cRect.width / 2
      const cy = cRect.top + cRect.height / 2
      const dist = Math.hypot(e.clientX - cx, e.clientY - cy)

      if (dist < 200) {
        const factor = Math.pow(1 - dist / 200, 1.6)
        gsap.to(char, {
          z: factor * 70,
          scale: 1 + factor * 0.2,
          duration: 0.2,
          ease: 'power2.out',
        })
      } else {
        gsap.to(char, { z: 0, scale: 1, duration: 0.55, ease: 'power3.out' })
      }
    })
  })
}
```